

4.8.4 多媒体相关计算问题

1. 图像容量计算

条件	示例
知道像素，位数	每个像素为16位，图像为640×480像素，求容量： $640 \times 480 \times 16 \div 8 = 614,400\text{B}$
知道像素，色数	640×480像素，256色的图像，求容量： $640 \times 480 \times \log_2(256) \div 8 = 307,200\text{B}$

2. 音频容量计算

容量 = 采样频率(Hz) × 量化/采样位数(位) × 声道数 ÷ 8

3. 视频容量计算

容量 = 每帧图像容量(Byte) × 每秒帧数 × 时间 + 音频容量 × 时间

上述计算中，图像中要理解色数的概念。要理解音频容量计算的原理，就是每个采样通道的采样次数 × 每次采样的位数 × 总的采用通道数。

视频就是一帧帧图像的组合，因此本质是求图像容量，当然要加上音频容量。

注意单位 B 和 b 的区别和换算，注意 K（大写，1024，存储时才用）和 k（小写，1000）的区别，注意结果单位。

5 程序设计语言基础知识

5.1 程序设计语言基本概念

解释和编译：都是将高级语言翻译成计算机硬件认可的机器语言加以执行。不同之处在于编译程序生成独立的可执行文件，直接运行，运行时无法控制源程序，效率高。而解释程序不生成可执行文件，可以逐条解释执行，用于调试模式，可以控制源程序，因为还需要控制程序，因此执行速度慢，效率低。

程序设计语言的特点：

1. Fortran语言(科学计算，执行效率高)
2. Pascal语言(为教学而开发的，表达能力强，Delphi)
3. C语言(指针操作能力强，高效)
4. Lisp语言(函数式程序语言，符号处理，人工智能)
5. C++语言(面向对象，高效)
6. Java语言(面向对象，中间代码，跨平台)
7. C#语言(面向对象，中间代码，.Net)
8. Prolog语言(逻辑推理，简洁性，表达能力，数据库和专家系统)

5.2 程序设计语言的基本成分

数据成分：指一种程序设计语言的数据和数据类型。数据分为常量（程序运行时不可改变）、变量（程序运行时可以改变）、全局量（存储空间在静态数据区分配）、局部量（存储空间在堆栈区分配）。数据类型有整型、字符型、双精度、单精度浮点型、布尔型等。

运算成分：指明允许使用的运算符及运算规则。包括算术运算、逻辑运算、关系运算、位运算等。

控制成分：指明语言允许表述的控制结构。包括顺序结构、选择结构、循环结构。

传输成分：指明语言允许的数据传输方式。如赋值处理、数据的输入输出等。

5.3 传值与传址

传值调用：将实参的值传递给形参，形参的改变不会导致调用点所传的实参的值改变。实参可以是合法的变量、常量和表达式。

传址调用：即引用调用，将实参的地址传递给形参，即相当于实参存储单元的地址引用，因此其值改变的同时就改变了实参的值。实参不能为常量，只能是合法的变量和表达式。

因此，在编程时，要**改变参数值，就传址**，不改变，就传值。

5.4 编译程序基本原理

编译程序对高级语言源程序进行编译的过程中，要不断收集、记录和使用源程序中一些相关符号的类型和特征等信息，并将其存入符号表中，编译过程如下：

词法分析：是编译过程的第一个阶段。这个阶段的任务是从左到右一个字符一个字符地读入源程序，即对构成源程序的字符流进行扫描然后根据构词规则识别单词(也称单词符号或符号)。

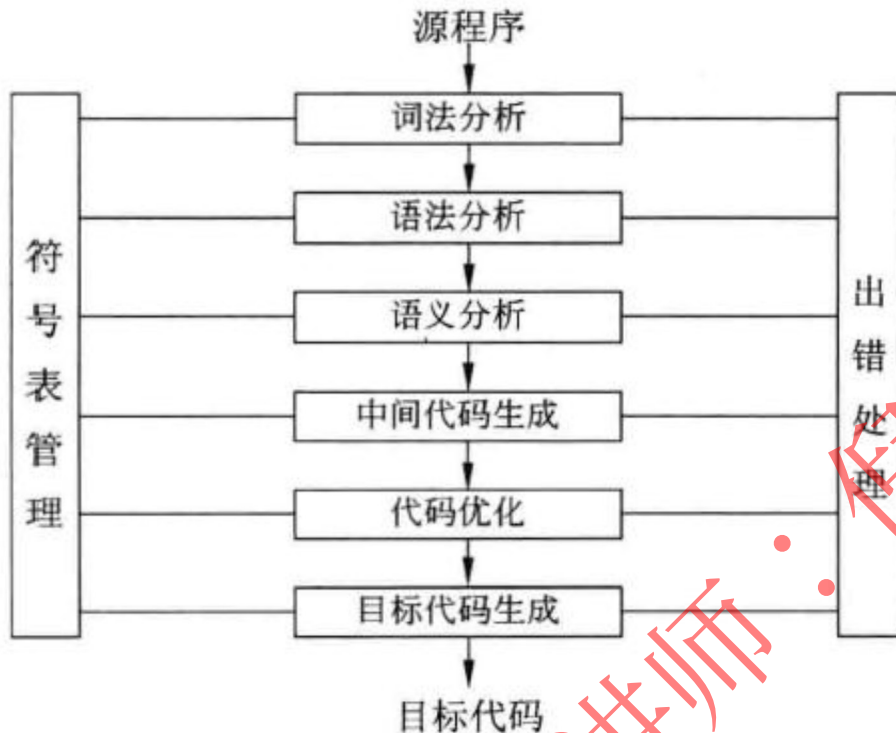
语法分析：是编译过程的一个逻辑阶段。语法分析的任务是在词法分析的基础上将单词序列组合成各类语法短语，如“程序”，“语句”，“表达式”等等。语法分析程序判断源程序在结构上是否正确。源程序的结构由上下文无关文法描述。

语义分析：是编译过程的一个逻辑阶段。语义分析的任务是对结构上正确的源程序进行上下文有关性质的审查，进行类型审查。如类型匹配、除法除数不为0等。又分为静态语义错误（在编译阶段能够查找出来）和动态语义错误（只能在运行时发现）。

中间代码和目标代码：中间代码是根据语义分析产生的，需要经过优化链接，最终生成可执行的目标代码。引入中间代码的目的是进行与机器无关的代码优化处理。常用的中间代码有后缀式（逆波

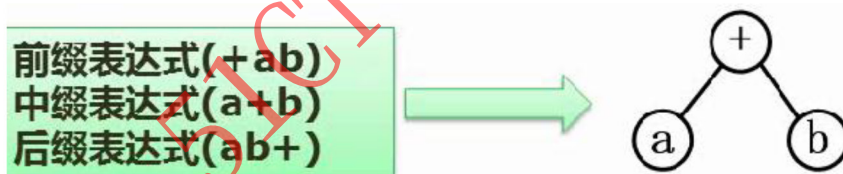
兰式)、三元式(三地址码)、四元式和树等形式。需要考虑三个问题(一是如何生成较短的目标代码;二是如何充分利用计算机中的寄存器,减少目标代码访问存储单元的次数;三是如何充分利用计算机指令系统的特点,以提高目标代码的质量)。

整个编译过程如下图所示:



声明语句,是函数声明、变量定义、宏定义等语句,存入符号表中供查询对应关系,可执行语句是逻辑流程代码,需要翻译成中间代码或目标代码。

5.5 表达式



主要掌握上述三种表达式即可,其实就是树的三种遍历,一般正常的表达式是中序遍历,即中缀表达式,根据其构造出树,再按题目要求求出前缀或后缀式。

简单求法: 后缀表达式是从左到右开始,先把表达式加上括号,再依次把运算符加到本层次的括号后面。

5.6 文法定义

定义如下图所示：要求了解四元组每个符号的含义。

一个形式文法是一个有序四元组 $G=(V, T, S, P)$ ，其中：

- 1) V ：非终结符。不是语言组成部分，不是最终结果，可理解为占位符。
- 2) T ：终结符。是语言的组成部分，是最终结果。 $V \cap T = \emptyset$
- 3) S ：起始符。是语言的开始符号。
- 4) P ：产生式。用终结符替代非终结符的规则。形如 $\alpha \rightarrow \beta$

终结符：最终结果，不能推导出其他元素。

非终结符：能够推导出其他元素。

产生式：即非终结符推导出终结符的公式。

闭包，概念如下图，一般考察闭包可以为0个的情况代入运算：

正则闭包： $A^+ = A^1 \cup A^2 \cup A^3 \cup \dots \cup A^n \cup \dots$ （也就是所有幂的组合）。

闭包： $A^* = A^0 \cup A^+$ （在正则闭包的基础上，加上 $A^0 = \{\epsilon\}$ ）。

例如 $a^* = \{a, aa, aaa, \dots, \epsilon\}$ ，而 $(ab)^* = \{ab, abab, ababab, \dots, \epsilon\}$

文法类型如下：

类 型	别 称	说 明	对应自动机
<u>0型</u>	短语文法	G 的每条产生式 $\alpha \rightarrow \beta$ 满足 α 属于 V 的正则闭包且至少含有一个非终结符，而 β 属于 V 的闭包	图灵机
<u>1型</u>	上下文有关文法	G 的任何产生式 $\alpha \rightarrow \beta$ 满足 $ \alpha \leq \beta $ ，仅仅 $S \rightarrow \epsilon$ 例外，但 S 不得出现在任何产生式右部	线性界限自动机
<u>2型</u>	上下文无关文法	G 的任何产生式为 $A \rightarrow \beta$ ， A 为非终结符， β 为 V 的闭包	非确定的下推自动机
<u>3型</u>	正规文法	G 的任何产生式为 $A \rightarrow \alpha B$ 或 $A \rightarrow \alpha$ ， α 属于非终结符的闭包， A 、 B 都属于非终结符	有限自动机

程序设计语言中的大部分语法都是上下文无关文法，当然语义上是相关的，要注意区分语法和语义。

5.7 正规式

正规式和正规集：

对于字母表 Σ ，其上的正规式及其表示的正规集可以递归定义如下。

- (1) ε 是一个正规式，它表示集合 $L(\varepsilon) = \{\varepsilon\}$ 。
- (2) 若 a 是 Σ 上的字符，则 a 是一个正规式，它所表示的正规集为 $\{a\}$ 。
- (3) 若正规式 r 和 s 分别表示正规集 $L(r)$ 和 $L(s)$ ，则：
 - ① $r|s$ 是正规式，表示集合 $L(r) \cup L(s)$ 。
 - ② $r \cdot s$ 是正规式，表示集合 $L(r)L(s)$ 。
 - ③ r^* 是正规式，表示集合 $(L(r))^*$ 。
 - ④ (r) 是正规式，表示集合 $L(r)$ 。

示例如下：

正 规 式	正 规 集
ab	字符串 ab 构成的集合
$a b$	字符串 a 、 b 构成的集合
a^*	由 0 个或多个 a 构成的字符串集合
$(a b)^*$	所有字符 a 和 b 构成的串的集合
$a(a b)^*$	以 a 为首字符的 a 、 b 字符串的集合
$(a b)^*abb$	以 abb 结尾的 a 、 b 字符串的集合

5.8 有限自动机

是进行词法分析的工具，其定义如下：

$M = (S, \Sigma, \delta, S_0, Z)$

- 1) S 是一个有限集，每个元素为一个状态
- 2) Σ 是一个有穷字母表，每个元素为一个输入字符
- 3) δ 是转换函数：是一个单值对照
- 4) S_0 属于 S ，是其唯一的初态
- 5) Z 是一个终态集（可空）

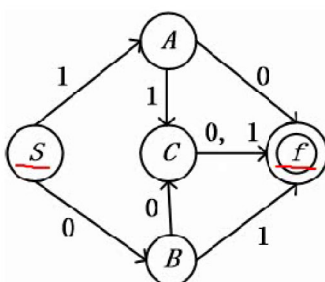
有限状态自动机可以形象地用状态转换图表示，设有限状态自动机：

$DFA = (\{S, A, B, C, f\}, \{1, 0\}, \delta, S, \{f\})$;

其中：

$\delta(S, 0) = B, \delta(S, 1) = A, \delta(A, 0) = f, \delta(A, 1) = C, \delta(B, 0) = C, \delta(B, 1) = f, \delta(C, 0) = f, \delta(C, 1) = f$

其状态转换图示例如下：



主要有五个符号集，由上图示例，可知用状态来表示十分清晰，由 S 输入一个 0，可得出 B，依次类推，一般考试，给出一个状态图，问能否构造出 001 这样的字符串，解决方法就是从起始 S 到终点 f 之间是否有一条路，权值为 001。本质就是有向图从起点到终点的遍历。

确定的有限自动机和不确定的有限自动机：输入一个字符，看是否能得出唯一的后继，若能，则是确定的，否则若得出多个后继，则是不确定的。

5.9 语法分析方法

自上而下语法分析：最左推导，从左至右。给定文法 G 和源程序串 r。从 G 的开始符号 S 出发，通过反复使用产生式对句型中的非终结符进行替换(推导)，逐步推导出 r。

递归下降思想：原理是利用函数之间的递归调用模拟语法树自上而下的构造过程，是一种自上而下的语法分析方法。

自下而上语法分析：最右推导，从右至左。从给定的输入串 r 开始，不断寻找子串与文法 G 中某个产生式 P 的候选式进行匹配，并用 P 的左部代替(归约)之，逐步归约到开始符号 S。

移进-规约思想：设置一个栈，将输入符号逐个移进栈中，栈顶形成某产生式的右部时，就用左部去代替，称为归约。很明显，这个思想是通过右部来推导出左部，因此是自下而上语法分析的核心思想。

程序设计语言基础知识

1、对于逻辑表达式“ $x \text{ and } y \text{ or not } z$ ”， and 、 or 、 not 分别是逻辑与、或、非运算，优先级从高到低为 not 、 and 、 or ， and 、 or 为左结合， not 为右结合，若进行短路计算，则_____。

- A. x 为真时，整个表达式的值即为真，不需要计算 y 和 z 的值
- B. x 为假时，整个表达式的值即为假，不需要计算 y 和 z 的值
- C. x 为真时，根据 y 的值决定是否需要计算 z 的值
- D. x 为假时，根据 y 的值决定是否需要计算 z 的值

答案：C

解析：在逻辑表达式运算中，短路计算是指通过逻辑运算符($\&\&$ 、 $\|$)左边表达式的值就能推算出整个表达式的值，不再继续执行逻辑运算符右边的表达式。对逻辑表达式“ $x \text{ and } y \text{ or not } z$ ”进行短路计算，若 x 为真时，则需要根据 y 的值决定是否需要计算 z 的值(即 y 为真则不需要计算 z 的值， y 为假则需要计算 z 的值)，来推算出整个表达式的值。

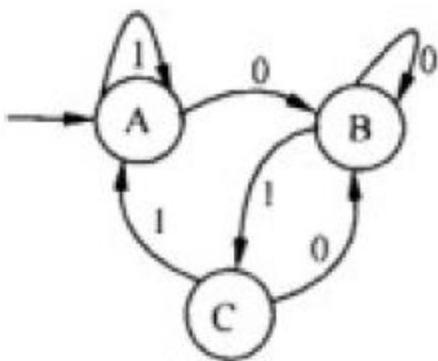
2、算术表达式 $x-(y+c)*8$ 的后缀式是_____($-$ 、 $+$ 、 $*$ 表示算术的减、加、乘运算，运算符的优先级和结合性遵循惯例)。

- A. $xyc8-+*$
- B. $xy-c+8*$
- C. $xy c 8*+-$
- D. $xy c + 8*-$

答案：D

解析：后缀式的求法很简单，首先给算术表达式每一层运算加上括号，即 $(x-((y+c)*8))$ ，再将运算符都提取到每一层表达式的后面，即 $(x((yc)+8)*)-$ ，最后去掉所有括号得 $xyz+8*-$ 。

3、下图所示为一个有限自动机(其中，A 是初态、C 是终态)，该自动机所识别的字符串的特点是_____。



- A. 必须以 11 结尾的 0、1 串
- B. 必须以 00 结尾的 0、1 串
- C. 必须以 01 结尾的 0、1 串
- D. 必须以 10 结尾的 0、1 串

答案：C

解析：有限自动机识别的是从初态到终态的字符串，也即从 A 到 C 的字符串，选项中考察以什么字符串结尾，因此只需要查看终态的输入即可，C 只能输入 1，因此最后一个字符必然是 1，在此基础上，反推到 B，只能输入 0，因此倒数第二个必然为 0，可知以 01 结尾的 0、1 串。

4、函数(过程)调用时，常采用传值与传地址两种方式在实参与形参间传递信息。以下叙述中，正确的是_____。

- A. 在传值方式下，将形参的值传给实参，因此，形参必须是常量或变量
- B. 在传值方式下，将实参的值传给形参，因此，实参必须是常量或变量
- C. 在传地址方式下，将形参的地址传给实参，因此，形参必须有地址
- D. 在传地址方式下，将实参的地址传给形参，因此，实参必须有地址

答案：D

解析：函数调用时基本的参数传递方式有传值与传地址两种。传值调用是将实参的值传给被调用函数的形参，因此实参可以是常量、变量、表达式或函数调用；传地址调用(或引用调用)的实质是将实参的地址传给被调用函数的形参，因此实参必须具有地址。

5、编译和解释是实现高级程序设计语言翻译的两种基本形式。以下关于编译与解释的叙述中，正确的是_____。

- A. 在解释方式下，对源程序不进行词法分析和语法分析，直接进行语义分析
- B. 在解释方式下，无需进行语法、语义和语义分析，而是直接产生源程序的目标代码
- C. 在编译方式下，必须进行词法、语法和语义分析，然后再产生源程序的目标代码
- D. 在编译方式下，必须先形成源程序的中间代码，然后再产生与机器对应的目标代码

答案：C

解析：在编译方式下，先将源程序翻译为等价的目标程序，源程序的翻译和目标程序的运行是完全独立的两个阶段；而解释方式下，对源程序的翻译和运行是结合在一起进行的，并不生成目标代码。编译过程基本上可以划分为词法分析、语法分析、语义分析、中间代码生成、代码优化和目标代码生成等几个阶段，其中，中间代码生成和代码优化不是必须的。在词法、语法、语义分析方面，编译方式和解释方式没有区别。

6. 可用于编写独立程序和快速脚本的语言是_____。

- A. Python B. Prolog C. Java D. C#

答案：A

解析：脚本语言只有 A，CD 是高级程序设计语言，B 是逻辑推理相关语言。

7. 语言 $L=\{a^m b^n | m \geq 0, n \geq 1\}$ 的正规表达式是_____。

- A. aa^*bb^* B. a^*bb^* C. aa^*b^* D. a^*b^*

答案：B

解析：由 $m \geq 0$ 可知，a 可以不出现，由 $n \geq 1$ 可知，b 必须至少出现一次，只有 B 满足条件，* 为闭包符号，表示该字符的 0 到 n 次幂。

8. 算术表达式 $(a-b)*c+d$ 的后缀式是_____（-、+、* 表示算术的减、加、乘运算，运算符的优先级和结合性遵循惯例）

- A. $a\ b\ c\ d\ -\ *+\$ B. $a\ b\ -\ c\ d\ *+\$
C. $a\ b\ -\ c\ *d+\$ D. $a\ b\ c\ -\ d\ *+\$

答案：C

解析：后缀式的求法很简答，先为每一层运算加上括号，转换为 $((a-b)*c)+d$ ，然后将所有的运算符提取到每一层括号的后面，转化为 $((ab-c)*d)+$ ，再去掉所有括号的 $ab-c*d+$ 。

9. 将高级语言源程序翻译成目标程序的是_____。

- A. 解释程序 B. 编译程序 C. 链接程序 D. 汇编程序

答案：B

解析：生成目标程序，是编译程序。

10. 在对程序语言进行翻译的过程中，常采用一些与之等价的中间代码表示形式，常用的中间代码表示不包括_____。

- A. 树 B. 后缀式 C. 四元式 D. 正则式

答案：D

解析：正则式是规则表达式，用来检索替换的条件表达式，与中间代码无关。

11. 以下关于程序错误的叙述中，正确的是_____。

- A. 编译正确的程序必然不包含语法错误
- B. 编译正确的程序必然不包含语义错误
- C. 除数为 0 的错误可以在语义分析阶段检查出来
- D. 除数为 0 的错误可以在语法分析阶段检查出来

答案：A

解析：编译正确的程序必然不包含词法、语法错误，因为词法和语法都是检查拼写和关键字，但是语义错误是无法完全检测出来的，和程序员的算法设计有关，只有运行时才能发现，例如除数为 0 只有在运行时真的出现了除数为 0 程序才能出现异常，而无法在编译阶段发现。

●以下关于解释程序和编译程序的叙述中，正确的是(12)

12. A. 编译程序和解释程序都生成源程序的目标程序
- B. 编译程序和解释程序都不生成源程序的目标程序
- C. 编译程序生成源程序的目标程序，而解释程序则不然
- D. 编译程序不生成源程序的目标程序，而解释程序反之

答案：C

解析：常考了；编译程序生成目标程序，不参与运行控制，运行效率高；解释程序则相反。

●以下关于传值调用与引用调用的叙述中，正确的是（13）

- ①在传值调用方式下，可以实现形参和实参间双向传递数据的效果
- ②在传值调用方式下，实参可以是变量，也可以是常量和表达式
- ③在引用调用方式下，可以实现形参和实参间双向传递数据的效果
- ④在引用调用方式下，实参可以是变量，也可以是常量和表达式

- 13.A. ①③ B. ①④ C. ②③ D. ②④

答案：C

解析：传值调用传值，无法修改实参值，也即不能双向传递数据，因为是传值，因此实参可以是常量、变量和表达式；传址因为是传递地址，因此实参不能是常量。

●在对高级语言源程序进行编译的过程中，为源程序中变量所分配的存储单元的地址属于(14)

- 14.A.逻辑地址 B.物理地址 C.接口地址 D.线性地址

答案：A

解析：编译过程中为源程序变量所分配的地址都是逻辑地址，在执行时会涉及到将逻辑地址转换为物理地址。

●以下关于语言 $L=\{a^nb^n|n\geq 1\}$ 的叙述中，正确的是（15）

15. A.可用正规式“ $aa*bb*$ ”描述，但不能通过有限自动机识别
B.可用正规式“ a^mb^m ”表示，但可用有限自动机识别
C.不能用正规式表示，但可用有限自动机识别
D.不能用正规式表示，也不能通过有限自动机识别

答案：D

解析：首先确定能否用正规式表示，在该语言表达式里， n 次方是一个 ≥ 1 的数，因此要求 a 和 b 出现的次数相等且至少出现一次，A 中 $*$ 表示的是任意次方，无法保证 a 和 b 出现次数相等，B 中 m 保证出现次数相等，但是 m 可以任意取值，都不符合，因此不能用正规式表示；

然后确定能否用有限自动机识别，在该表达式中 n 是不确定的，也就是说它并没有终态，不知道何时应该结束，因此不能用有限自动机识别。

●编译过程中，对高级语言程序语句的翻译主要考虑声明语句和可执行语句。对声明语句，主要是将所需要的信息正确地填入合理组织的（16）中；对可执行语句，则是（17）

16. A.符号表 B.栈 C.队列 D.树
17. A.翻译成机器代码并加以执行 B.转换成语法树
C.翻译成中间代码或目标代码 D.转换成有限自动机

答案：A C

解析：声明语句，就是函数声明、变量定义、宏定义等语句，存入符号表中供查询对应关系，可执行语句就是逻辑流程代码，需要翻译成中间代码或目标代码。

● 程序运行过程中常使用参数在函数（过程）间传递信息，引用调用传递的是实参的（18）

- (18) A 地址 B 类型 C 名称 D 值

答案：A

解析：常识了，引用传递实参地址，传值传递实参值。

● 已知文法 $G: S \rightarrow A0|B1, A \rightarrow S1|1, B \rightarrow S0|0$ ，其中 S 是开始符号。从 S 出发可以推导出 (19)。

- (19) A、所有由 0 构成的字符串
B、所有由 1 构成的字符串
C、某些 0 和 1 个数相等的字符串
D、所有 0 和 1 个数不同的字符串

答案：C

解析：将 AB 代入 S 公式转换得 $S \rightarrow S1|10|S0|01$ ，此公式中只有 S，没有其他符号，可以看出 0 和 1 的个数相等，并且 | 符号下有 01 和 10 同时出现，因此从 S 出发推导出的公式 0 和 1 必然相等且同时出现。

● 算术表达式 $a+(b-c)*d$ 的后缀式是(20) (-、+、*表示算术的减、加、乘运算，字符的优先级和结合性遵循惯例)

- (20) A. $c-d*a$ B. $abc-d*+$ C. $ab+c-d*$ D. $abcd-*+$

答案：B

解析：后缀式的求法很简单，从左到右将每一个运算的表达式加上括号，即为 $((a)+((b-c)*d))$ ，再将运算符号移到对应层次括号后面即 $a((bc)-d)*+$ ，将所有括号去掉，就是 $abc-d*+$ 。在文老师视频课程中有详细讲解。

● 将高级语言程序翻译为机器语言程序的过程中，常引入中间代码，其好处是 (21)

- (21) A、有利于反编译处理 B、有利于进行与机器无关的优化处理
C、尽早发现语法错误 D、可以简化语法和语义分析

答案：B

解析：从编译各个阶段可以看出，生成中间代码后是代码优化阶段，因此，中间代码的好处是进行代码的优化，语法和语义在之前就完成，与中间代码已经无关。

● 对高级语言源程序进行编译的过程中，有穷自动机(NFA 或 DFA)是进行(22)的适当工具。

- (22) A. 词法分析 B. 语法分析 C. 语义分析 D. 出错处理

答案：A

解析：DFA 和 NFA 都是识别字符串的集合，很明显，对字符进行分析识别，属于词法分析。

● 弱类型语言（动态类型语言)是指不需要进行变量/对象类型声明的语言。(23)属于弱类型语言。

- (23) A. java B. c/c++ C. Python D. C#

答案：C

解析：常识题；由题意，不需要进行变量声明就直接可以使用的，是脚本语言 Python。

●以下程序设计语言中， (24) 更适合用来进行动态网页处理。

(24) A.HTML B. LISP C.PHP D.JAVA/C++

答案：C

解析：动态网页相关的语言有 ASP、JSP 和 PHP，静态网页语言是 HTML。

●在引用调用方式下进行函数调用是将 (25) 。

(25) A. 实参的值传递给形参 B. 实参的地址传递给形参
C. 形参的值传递给实参 D. 形参的地址传递给实参

答案：B

解析：传引用的调用方式，是将实参的地址传递给形参，改变形参将会同时改变实参的值。

●编译程序对高级语言源程序进行编译的过程中，要不断收集、记录和使用源程序中一些相关符号的类型和特征等信息，并将其存入 (26) 中。

(26) A. 符号表 B. 哈希表 C. 动态查找表 D. 栈和队列

答案：A

解析：从题意描述也可以立即，相关符号类型和特征等信息应该存入符号表中，实际上，在编译过程中符号表是一种记录函数名、变量名、地址等信息的文件，用于在编译时核对。

●以下关于实现高级程序设计语言的编译和解释方式的叙述中，正确的是 (27) 。

(27) A. 在编译方式下产生源程序的目标程序，在解释方式下不产生
B. 在解释方式下产生源程序的目标程序，在编译方式下不产生
C. 编译和解释方式都产生源程序的目标程序，差别是优化效率不同
D. 编译和解释方式都不产生源程序的目标程序，差别在是否优化

答案：A

解析：编译方式和解释方式的主要区别就是是否生成目标程序，以及是否能控制程序运行，编译方式生成目标程序，不能控制程序运行，效率高，解释方式则相反。

●大多数程序设计语言的语法规则用 (28) 描述即可。

- (28) A. 正规文法 B. 上下文无关文法
 C. 上下文有关文法 D. 短语结构文法

答案：B

解析：常考了，程序设计语言语法是上下文无关的，但是语义是相关的。

●在某 C/C++程序中，整型变量 a 的值为 0 且应用在表达式“c=b/a”中，则最可能发生的情形是(29)。

- (29) A. 编译时报告有语法错误 B. 编译时报告有逻辑错误
 C. 运行时报告有语法错误 D. 运行时产生异常

答案：D

解析：程序员都知道，这种情况编译是检查不出来的，会在程序运行时产生异常，而且运行时不会报告有无语法错误，语法检查是在编译阶段完成的，运行阶段的错误都直接产生异常。

30、属于面向对象、解释型程序设计语言的是 ()。

- A. XML B. Python C. Prolog D. C++

答案：A

解析：注意共三个条件，面向对象、解释型、程序设计语言，B 是脚本语言，C 不是面向对象，D 是编译型语言，只有 A 符合三个条件，XML 不生成目标代码，直接加载解释执行。

31、算术表达式“(a-b)*(c+d)”的后缀式是 ()。

- A. ab-cd+* B. abcd-*+ C. ab-*cd+ D. ab-c+d*

答案：A

解析：后缀表达式是从左到右开始，先把表达式加上括号为((a-b)*(c+d))，再依次把运算符加到本层次的括号外面得出 ab-cd+*。

32、将高级语言源程序翻译成机器语言程序的过程中，常引入中间代码。以下关于中间代码的叙述中，不正确的是 ()。

- A. 中间代码不依赖于具体的机器。
B. 使用中间代码可提高编译程序的可移植性
C. 中间代码可以用树或图表示

D. 中间代码可以用栈和队列表示

答案：D

解析：引入中间代码一方面也是方便移植，因此其不依赖于具体的机器，可以用语法分析树或图表示，D选项自相矛盾，栈和队列是截然相反的线性数据结构，适用于截然相反的数据序列，不可能通用。

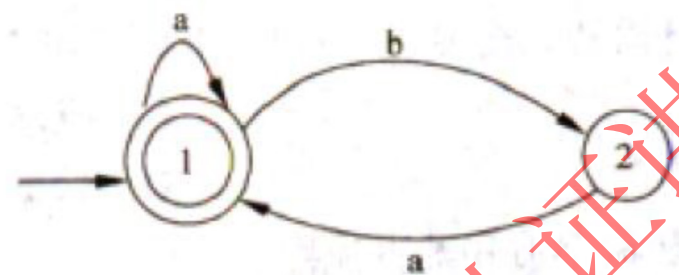
33、对高级语言源程序进行编译的过程可以分为多个阶段，分配寄存器的工作在（）阶段进行。

A. 词法分析 B. 语法分析 C. 语义分析 D. 目标代码生成

答案：D

解析：前三个都是错误检查分析，不涉及到具体存储分配工作，目标代码生成阶段应考虑直接影响到目标代码速度的三个问题：一是如何生成较短的目标代码；二是如何充分利用计算机中的寄存器，减少目标代码访问存储单元的次数；三是如何充分利用计算机指令系统的特点，以提高目标代码的质量。

34、以下关于下图所示有限自动机的叙述中，不正确的是（）。



- A. 该自动机识别的字符串中 a 不能连续出现
- B. 自动机识别的字符串中 b 不能连续出现
- C. 该自动机识别的非空字符串必须以 a 结尾
- D. 该自动机识别的字符串可以为空串

答案：A

解析：由图中可知，1 即是初态又是终态，并且字符 a 形成环路，因此 a 可以连续出现。

35、对于大多数通用程序设计语言，用（）描述其语法即可。

- A. 正规文法 B. 上下文无关文法
- C. 上下文有关文法 D. 短语结构文法

答案：B

解析：程序设计语言中语法是上下文无关的，只是语义上下文有关，要牢记。